

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

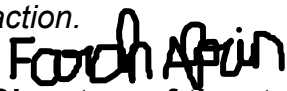
COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

I M. Farah Afrin 192511434 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

QUESTIONS:4

Total Marks:40

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

aQuestions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS, DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions	Logical solutions with adequate	Basic solutions with weak	Incorrect or no solution provided

		with strong justification	justification	justification	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Video Conferencing Quality Degradation

Scenario

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

1. Is the Problem Primarily Transport-Layer or Application-Layer?

The symptoms — voice distortion, frozen frames, delayed communication — combined with the measured 8% packet loss and 180 ms jitter, point mainly to the transport layer, specifically to how the underlying protocol (or a poor protocol choice) handles loss and timing for real-time media. The evidence for this diagnosis:

- Jitter (180 ms) is a transport/network-level metric — it measures variation in inter-packet arrival time caused by queuing and route changes, not something the application layer controls directly.
- Packet loss (8%) is extremely high for real-time media; ITU-T guidance treats even 1% loss as noticeable and >5% as severely degrading for voice/video. This magnitude of loss almost always originates below the application — congested links, insufficient QoS prioritization, or a transport protocol reacting badly to loss.
- If the platform is (incorrectly) using a TCP-based transport for the live audio/video streams, TCP's reliable-delivery guarantee would force retransmission and in-order delivery of media packets that are already late — this produces exactly the 'frozen frame, catch-up' behaviour described, because TCP will stall newer frames behind a retransmission of an old, now-useless one.

That said, application-level buffering strategy is a secondary contributing factor, not the root cause: if the jitter buffer is too small, 180 ms of jitter will cause audible dropouts even over a well-behaved transport; if it is too large, it adds delay on top of network delay. The recommendation below therefore treats this as primarily a transport-layer problem with an application-layer buffering component that must be tuned together.

2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia has fundamentally different delivery requirements from typical data transfer, which is why the transport choice matters so much here:

Requirement	File/Web Transfer (TCP fits)	Live Audio/Video (needs UDP-style transport)
Timeliness	Not critical — data can arrive late	Critical — a late packet is worse than a lost one
Ordering	Must be strict, in-order	Loose — a few out-of-order/lost frames

		are tolerable
Retransmission	Required for correctness	Counter-productive — retransmitted data usually arrives too late to render
Acceptable loss	Effectively zero	Small, bounded loss (a few %) is acceptable and concealable
Latency budget	Seconds are fine	ITU-T G.114 recommends < 150 ms one-way for acceptable voice quality

Because TCP enforces reliable, in-order delivery, any loss forces it to pause and retransmit, and its congestion-control algorithm reduces the sending rate after loss — both are exactly wrong for real-time media, where a stale audio/video frame is worthless and should be dropped and concealed, not resent. This is why real-time conferencing protocols are built on UDP (carrying RTP/RTCP), which delivers packets as soon as they arrive, tolerates loss, and leaves recovery decisions (skip, conceal, request a key-frame) to the application, which can make a timing-aware choice that TCP cannot.

3. Recommended Improvements

Protocol-Level

- Confirm the platform uses UDP-based RTP/SRTP for media (not TCP or a TCP-tunnelled fallback); if the client is falling back to TCP (common when UDP is blocked by a firewall), open/allow the required UDP port ranges.
- Enable Forward Error Correction (FEC) and/or NACK-based selective retransmission at the RTP layer so isolated losses are concealed or repaired without stalling the whole stream, unlike blanket TCP retransmission.
- Deploy DSCP/QoS marking (e.g. EF for voice, AF41 for video) on the network so conferencing traffic is prioritized over bulk/background traffic during peak hours, directly reducing queuing-induced jitter and loss.
- Use adaptive bitrate / congestion control designed for real-time media (e.g. Google Congestion Control used by WebRTC) so the sender backs off bandwidth smoothly instead of the video simply freezing.

Application-Level

- Tune the receiver's adaptive jitter buffer to track the measured 180 ms jitter rather than using a fixed buffer — too small causes underruns/glitches, too large adds unnecessary delay.
- Enable packet-loss concealment (PLC) for audio and frame-skipping/error-concealment for video so a lost packet degrades quality gracefully instead of freezing the call.
- Route through the nearest regional media relay/SFU (Selective Forwarding Unit) to shorten the path and reduce the chance of congestion-induced loss during peak hours.
- Monitor and auto-switch codecs/resolution (simulcast) based on real-time loss/jitter telemetry so quality degrades in a controlled way rather than the session breaking down.

Case Study 2 — High DNS Resolution Time (Global Web Services)

Scenario

A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

1. Reasons Behind the High DNS Resolution Time

- Geographic distance to authoritative servers: if the authoritative DNS servers are centralized in one region, employees on other continents incur a long round-trip time purely from physical distance/propagation delay before they even reach the name server.
- Deep referral chain / recursive lookup depth: a full recursive resolution (root → TLD → authoritative) with no caching in between forces multiple sequential round trips, each adding latency, especially if any hop is far away.
- Low or misconfigured TTL values: if DNS records have very short Time-To-Live values, resolvers are forced to repeat the full lookup far more often instead of serving a cached answer, multiplying the number of 800 ms lookups users experience.
- Single point of resolution (no Anycast): if all queries funnel to one primary server/location rather than being routed to the topologically nearest instance, every query pays the full distance penalty regardless of where the user is.
- Resolver-side congestion or overload: a single DNS server handling global query volume can queue requests during peak hours, adding processing delay on top of network delay.
- Lack of client-side/browser DNS pre-fetching: without pre-fetching, the DNS lookup sits directly in the critical path of the very first request, so its full cost is felt before the homepage can even begin loading.

2. Proposed DNS Architecture (Target: < 50 ms Lookup)

Technique	How It Reduces Lookup Time
Anycast DNS	The same DNS IP address is announced from multiple geographically distributed points-of-presence; BGP routes each user's query to the nearest instance automatically, cutting propagation delay from hundreds of ms to typically single-digit/low-double-digit ms.
DNS Pre-fetching	The browser/OS resolves domains it predicts will be needed (e.g. links on the current page, known internal services) in the background before the user actually requests them, so the 'visible' lookup time on the critical path drops to near 0 ms for pre-fetched names.
TTL Optimization	Setting an appropriately longer TTL (e.g. 300–3600 s for stable records)

	lets resolvers and clients reuse cached answers far more often, so only a small fraction of requests pay the full resolution cost; TTL is kept short only for records that must change quickly (e.g. failover records).
Geo-distributed secondary/authoritative servers	Placing authoritative name servers in each major region (in addition to Anycast) ensures that even a cache miss resolves against a nearby server rather than crossing continents.
Local caching resolvers (e.g. on-premises or per-region recursive resolvers)	Corporate offices/regions run their own caching resolver, so repeated lookups for the same popular internal/external domains are served from local memory almost instantly.

Combined, this architecture works as follows: a user's query first hits a local caching resolver (near-instant if cached); on a cache miss, Anycast routes the query to the nearest of several geographically distributed authoritative servers; pre-fetching ensures many lookups happen off the critical path entirely; and sensible TTLs keep the cache-hit rate high so the 800 ms 'cold' lookup path is rarely exercised. Because Anycast additionally provides multiple redundant instances of the same service address, continuous availability during a server failure is achieved automatically: BGP simply withdraws the route to the failed instance and traffic converges on the next-nearest healthy one, without any client-visible change of IP address.

3. Advantages and Limitations of the Proposed Architecture

Aspect	Advantages	Limitations
Performance	Anycast + regional resolvers cut round-trip distance dramatically; pre-fetching removes lookups from the critical path entirely	Anycast performance depends on BGP routing quality; poorly peered ISPs may still route to a farther PoP
Scalability	Additional PoPs can be added without changing the service's public IP address; load is naturally distributed by proximity	Managing many geographically distributed authoritative servers increases operational and synchronization complexity
Availability / Fault tolerance	Anycast automatically reroutes around a failed PoP via BGP convergence with no client reconfiguration; multiple authoritative servers avoid a single point of failure	BGP convergence is not instantaneous — a brief window of packet loss/timeouts can occur during failover; misconfigured Anycast can cause asymmetric routing issues
Consistency / Freshness	Longer TTLs reduce load and latency	Longer TTLs mean DNS-based failover or record changes propagate more slowly, so TTL must be tuned per record type (long for stable records, short for failover-critical ones)
Cost / Complexity	Significant, measurable UX improvement for a global user base	Higher infrastructure cost (multiple PoPs, monitoring, BGP management) and requires DNS/network engineering expertise to operate correctly

Case Study 3 — Centralized DNS Bottleneck at Global Scale

Scenario

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

1. Impact of Centralized DNS Architecture on Application Performance

- Single point of congestion: every user worldwide funnels through one server, so during high-traffic events (promotions) the server's query queue grows, adding processing/queuing delay on top of network latency — directly explaining the 800 ms figure.
- Distance penalty for all remote users: users far from the single server's location always pay the full propagation delay; there is no 'nearest instance' to shorten the path, unlike a distributed design.
- Single point of failure: if the primary server becomes overloaded or fails, DNS resolution for the entire global user base is affected simultaneously — there is no automatic failover path, risking a total outage rather than a localized slowdown.
- Poor scalability under load spikes: promotional events cause sudden, large increases in concurrent users; a single server has a hard capacity ceiling, so performance degrades sharply exactly when it matters most (peak business value).
- Cascading effect on application performance: since DNS resolution sits before any application request can even begin, an 800 ms DNS delay is added in full to every page load / API call, amplifying perceived slowness across the whole platform.

2. Distributed DNS Solution Design

A distributed architecture should combine the following layers:

1. Anycast-based authoritative DNS: deploy the authoritative name servers behind a single Anycast IP announced from multiple points-of-presence (PoPs) across continents, so each query is BGP-routed to the nearest healthy instance rather than a fixed single server.
2. Geographically distributed secondary servers: maintain multiple synchronized authoritative servers (primary + secondaries) in different regions, so no single server is a bottleneck or single point of failure; use standard zone-transfer/replication to keep them consistent.
3. Multi-tier caching: rely on recursive resolvers at the ISP/regional level and client-side OS/browser caching, backed by well-tuned TTLs, so the large majority of repeat queries never reach the authoritative layer at all.

4. Health-checked load balancing / DNS failover: integrate active health checks so that if a PoP or backend service becomes unhealthy, its records are automatically withdrawn or redirected (e.g. via short-TTL failover records) to a healthy region.
5. Elastic capacity for peak events: provision the distributed nodes to auto-scale (or over-provision ahead of known promotional events) so query-handling capacity grows with demand instead of queuing at a fixed-capacity single server.

With this design, a user's query is answered by the nearest of several redundant, synchronized servers; if any one instance fails or is overloaded, Anycast/BGP convergence and health checks redirect traffic to the next-nearest healthy instance automatically, without requiring any change on the client side — turning the earlier single point of failure into a resilient, horizontally scalable service.

3. Influence of TTL, Anycast Routing, and DNS Caching

Factor	Effect on Performance	Effect on Fault Tolerance
TTL Values	Longer TTL → higher cache-hit rate → far fewer full lookups → lower average resolution time and lower load on authoritative servers	Longer TTL → slower propagation of failover/record changes, so critical failover records should use short TTLs even though this slightly increases their query load
Anycast Routing	Routes each query to the topologically nearest PoP, minimizing propagation delay regardless of where in the world the user is	Automatically reroutes traffic away from a failed or unreachable PoP once BGP converges, without any change to the service's public IP or client configuration
DNS Caching (multi-tier: OS, browser, ISP resolver)	Absorbs the vast majority of repeat queries locally, so most user-visible lookups are effectively 0 ms after the first resolution	Reduces load reaching the authoritative tier, indirectly improving resilience by keeping the authoritative servers from being overwhelmed even if one region experiences a traffic spike

Together, these three factors form a layered defence: caching minimizes how often the authoritative layer is even contacted; Anycast minimizes the delay and maximizes resilience when it is contacted; and correctly tuned TTLs balance the trade-off between caching efficiency (favouring long TTLs) and rapid failover response (favouring short TTLs) for the specific records that need it.

Case Study 4 — Latency Spike in an Electronic Stock Trading Platform

Scenario

A financial institution operates an electronic stock trading platform where

thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Three TCP-Related Factors Behind the Latency Increase

a) Flow Control (Receive Window Exhaustion)

TCP flow control uses the receiver's advertised window to prevent a fast sender from overwhelming a slow receiver's buffer. At market open, order volume spikes sharply; if the receiving application (matching engine or gateway) cannot drain its socket buffer as fast as orders arrive, its advertised window shrinks toward zero. The sender then must pause and wait (a 'zero-window' condition) before sending more data, directly adding delay to order submission and acknowledgment — exactly the 'excessive buffering' observed.

b) Nagle's Algorithm (Send-Side Coalescing Delay)

Nagle's algorithm delays sending small segments in order to coalesce them into larger ones, to improve efficiency on the sender side. Trading messages (order requests, acknowledgments) are typically small and latency-critical. If Nagle's algorithm is enabled, the stack may hold a small outgoing message waiting either for more data to batch with it or for an ACK of previously sent data (interacting badly with Delayed ACK on the receiver side) — this classic 'Nagle + Delayed ACK' interaction can by itself introduce tens of milliseconds of added latency per message, which lines up closely with the observed rise from 1 ms to 40 ms.

c) SYN Queue Limitations (Connection Establishment Backlog)

Each new TCP connection begins with a SYN arriving at the listening socket's SYN queue (backlog) before the three-way handshake completes and the connection is accepted by the application. At market open, many trading clients/sessions attempt to (re)connect simultaneously; if the configured SYN backlog (and/or the accept queue) is too small, incoming SYNs are queued, delayed, or dropped and must be retried, lengthening connection establishment time — consistent with the engineers' observation of 'longer connection establishment times' and contributing to the retransmissions seen (retransmitted SYNs when the queue overflows).

2. How Each Factor Affects High-Frequency Transaction Processing

Factor	Effect on Transaction Processing
Flow control (window exhaustion)	Order/acknowledgment messages queue up waiting for buffer space instead of being transmitted immediately, directly delaying trade confirmations during the highest-value trading window (market open) and risking order timeouts.
Nagle's algorithm	Small, latency-critical order messages are artificially delayed by the stack itself even when the network has spare capacity, adding tens of milliseconds of pure software-induced latency per round trip — unacceptable for high-frequency trading, where microsecond/low-millisecond response times are the norm.

SYN queue limitations

New or reconnecting trading sessions experience delayed or failed connection setup during the peak connection-attempt surge at market open, pushing back the point at which a client can begin submitting orders at all, and increasing retransmitted SYNs which further congests the link.

3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm on trading connections by setting `TCP_NODELAY` on the sockets used for order submission/acknowledgment, so small latency-critical messages are sent immediately instead of being batched.
- Disable or tune Delayed ACK (e.g. `TCP_QUICKACK` on Linux) on the peer side so acknowledgments are not held back and interacting with a disabled Nagle's algorithm to compound delay.
- Increase the receive/send socket buffer sizes (`SO_RCVBUF` / `SO_SNDBUF`) and enable TCP window scaling so the effective window at market-open traffic volumes does not collapse to zero, keeping data flowing under flow control.
- Increase the SYN backlog and accept queue sizes (e.g. Linux `net.core.somaxconn` and the `listen()` backlog parameter, plus `net.ipv4.tcp_max_syn_backlog`) so a surge of simultaneous connection attempts at market open can be queued and accepted without being dropped or retried.
- Enable SYN cookies as a safety net for handling connection-request bursts without exhausting the SYN queue, while still allowing legitimate high-volume reconnections to succeed.
- Pre-establish and keep-alive persistent TCP connections (connection pooling) for known trading clients ahead of market open, so the handshake cost is paid once, off the critical path, rather than repeatedly during the highest-traffic window.
- Consider a less conservative initial congestion window / a congestion-control algorithm suited to low-latency bursts (e.g. BBR) to reduce ramp-up delay for short-lived, latency-sensitive trading connections.
- Monitor and tune these parameters specifically for the market-open traffic profile (short burst of very high connection and message rate), validating improvements with the same retransmission/latency metrics that revealed the problem.

1. Analysis of Network Problems in Cloud-Based Video Conferencing

Introduction

In the given case, employees of a multinational IT company are facing problems such as voice distortion, frozen video, and delayed conversations while using a cloud-based video conferencing platform. The network shows **8% packet loss, 180 ms jitter, and unstable bandwidth utilization** during peak hours.

These values clearly show that the communication system is under stress when many employees are using the network at the same time. The problem is mainly related to the way real-time data is transported over the network, but the application's buffering and quality-control mechanisms also have an important role.

1. Is the Problem Due to TCP/UDP or the Application Layer?

The problem can be considered **mainly a transport/network-related issue, with some contribution from the application layer.**

Video conferencing normally uses **UDP for real-time audio and video transmission** because speed is more important than delivering every packet perfectly. UDP does not wait for lost packets to be retransmitted. This helps keep communication fast.

However, UDP has one major disadvantage: it does not guarantee packet delivery. In this case, the network is already losing **8% of the packets**. Some portions of audio and video will therefore never reach the receiver. This explains the distorted voice and missing or frozen video frames.

The **180 ms jitter** is another major problem. Jitter means that packets are not reaching the receiver at a regular time. For example, one packet may arrive quickly while the next one arrives much later. If the application cannot handle this variation properly, the user experiences breaks in audio and video.

If the application uses TCP for the media stream, the situation can become worse because TCP retransmits lost packets. Although this improves reliability, waiting for retransmissions can introduce additional delay. In a live meeting, receiving an old packet several seconds later is often less useful than continuing with the current conversation.

Therefore, the main issue is **network congestion and unsuitable handling of real-time traffic**, rather than simply saying "UDP is bad" or "TCP is bad."

2. Why This Matters for Real-Time Multimedia

Real-time applications have different requirements compared with applications such as email or file downloading.

For example, if a file download loses a packet, the system can simply request that packet again. The user normally does not notice the small delay.

A video meeting is different. If a person says something and the audio reaches the other person 500 milliseconds later, the conversation becomes uncomfortable. Similarly, if a video packet is lost, the application cannot keep waiting for that packet forever because newer frames are already being generated.

The important requirements for video conferencing are:

1. **Low delay** – speech and video should reach the other person quickly.
2. **Low jitter** – packets should arrive at reasonably regular intervals.
3. **Low packet loss** – excessive loss produces missing audio and video.
4. **Stable bandwidth** – enough capacity must be available throughout the meeting.
5. **Continuous playback** – the application should avoid unnecessary freezing and buffering.

In this situation, **8% packet loss and 180 ms jitter are much higher than what a good real-time communication system would normally want**. The fluctuating bandwidth also suggests that network congestion is occurring during peak business hours.

This is why simply switching from UDP to TCP would not solve the problem. TCP might improve reliability, but its retransmission mechanism can increase delay. A better solution is to use a real-time transport mechanism and allow the application to intelligently handle packet loss and changing network conditions.

3. Recommended Improvements

A. Transport and Network-Level Improvements

Use UDP-based media transmission:

Audio and video should preferably continue using UDP-based real-time transport. It avoids unnecessary waiting for retransmission of old packets.

Introduce congestion control:

The sender should continuously monitor the network. When congestion increases, it should reduce the amount of data being sent instead of continuing at the same rate.

Give priority to voice and video:

The company can configure **Quality of Service (QoS)** so that real-time audio and video packets receive higher priority than activities such as large downloads or software updates.

Use Forward Error Correction (FEC):

Some additional redundant information can be transmitted along with the media. If a few packets are lost, the receiver may recover the missing information without waiting for retransmission.

Improve available bandwidth:

Since the problem occurs during peak hours, the company should also check whether its

internet connection or internal network capacity is sufficient for the number of simultaneous meetings.

B. Application-Level Improvements

Adaptive bitrate:

The application should automatically reduce video quality when the available bandwidth falls. For example, instead of allowing the video to completely freeze, it can temporarily lower the resolution.

Adaptive jitter buffer:

A small amount of buffering can smooth out irregular packet arrival. The buffer should not be unnecessarily large because a very large buffer itself introduces delay.

Packet-loss concealment:

For audio, the application can estimate small missing portions instead of producing an obvious interruption every time a packet is lost.

Give audio higher priority:

During severe network congestion, maintaining understandable speech is more important than maintaining high-definition video. The application can reduce video quality while keeping the audio relatively stable.

Continuous network monitoring:

The system should monitor packet loss, jitter, latency, and bandwidth continuously. Based on these values, it can automatically change the quality of the meeting.

Practical Example

Consider a meeting in which 100 employees are connected during the company's peak working hours. Suddenly, several employees start downloading large files. The available bandwidth decreases and packet loss increases.

Instead of allowing the video to continue at the same high bitrate, an improved conferencing application would detect the change. It could reduce the video resolution, give priority to audio packets, adjust the jitter buffer, and use error-recovery techniques for some lost packets.

As a result, the participants may notice slightly lower video quality, but the **conversation remains understandable and continuous**. This is much better than having high-quality video that repeatedly freezes.

Conclusion

From the given situation, the problem is mainly caused by **network congestion, high packet loss, and high jitter**, while the application's buffering and media adaptation can either reduce or worsen the effect.

For real-time communication, **UDP-based transport is generally more suitable than traditional TCP** because it avoids waiting for retransmission of delayed packets. However,

UDP alone cannot solve the problem. It should be supported by **congestion control, QoS, Forward Error Correction, adaptive bitrate, intelligent jitter buffering, and packet-loss concealment**.

Therefore, the best solution is not simply to replace one protocol with another. The company should improve both the **network/transport handling and the application's ability to adapt to changing network conditions**. This will provide smoother audio, fewer frozen frames, and more natural communication during peak business hours.

2. Analysis and Redesign of DNS Infrastructure

Introduction

A multinational software company provides web services to users and employees located across different continents. However, users are experiencing a noticeable delay before the application homepage starts loading. The network analysis shows that the **average DNS lookup time is around 800 ms**, which is very high for a web application.

DNS is responsible for converting a domain name into the IP address of the server hosting the application. If DNS resolution itself takes 800 ms, the browser has to wait before it can even start communicating with the web server. Therefore, improving the DNS infrastructure can significantly improve the initial page-loading experience.

The proposed solution is to combine **Anycast DNS, DNS caching and pre-fetching, and proper TTL optimization**, along with redundant DNS servers distributed across different geographical locations.

1. Reasons for the High DNS Resolution Time

There can be several reasons for the unusually high DNS lookup time.

1. Geographical Distance

The company operates across multiple continents. If users in Asia are querying a DNS server located in North America, the request has to travel a long physical distance. This increases network round-trip time.

2. Slow or Distant Recursive DNS Resolver

The user's DNS request may be handled by a recursive resolver that is overloaded or geographically far away. The resolver may need to contact several DNS servers before obtaining the final IP address.

3. Multiple DNS Queries

DNS resolution may involve several steps:

Client → Recursive Resolver → Root Server → TLD Server → Authoritative DNS Server

If the required information is not already cached, each step can add some delay.

4. Low or Poorly Selected TTL Values

If DNS records have very short TTL values, cached records expire frequently. This forces recursive DNS servers to perform fresh lookups more often, increasing DNS traffic and resolution time.

5. DNS Server Overload

If the company's authoritative DNS servers receive a large number of requests and have limited capacity, response times can increase, particularly during peak periods.

6. Network Congestion and Packet Loss

DNS queries use network communication like any other service. Congestion, packet loss, or retransmissions can delay DNS responses.

Therefore, the **800 ms average lookup time is probably not caused by a single issue**. It is more likely the result of geographical distance, poor DNS server placement, cache misses, and overloaded or inefficient DNS infrastructure.

2. Proposed DNS Architecture

The company should use a **globally distributed, redundant DNS architecture based mainly on Anycast DNS**, supported by DNS caching, pre-fetching, and suitable TTL values.

Proposed Architecture

User Device



Nearest Anycast DNS Resolver



Recursive DNS Cache



Distributed Authoritative DNS Servers



Application Server

The DNS infrastructure should have servers in major regions such as **Asia, Europe, North America, and other important user locations**.

A. Anycast DNS

In Anycast DNS, multiple DNS servers located at different geographical locations use the same IP address.

For example:

- DNS Server A → Asia
- DNS Server B → Europe

- DNS Server C → North America
- DNS Server D → Australia

All of them advertise the same Anycast address.

When a user sends a DNS request, network routing normally directs the request toward an appropriate nearby/low-cost Anycast location.

This reduces the physical distance between the user and the DNS service and can significantly reduce lookup latency.

B. DNS Caching

Caching should be used at recursive DNS resolvers.

Once a DNS record is obtained, the resolver can store it temporarily. If another employee requests the same domain before the record expires, the resolver can return the cached answer instead of performing the entire DNS resolution process again.

This can make subsequent DNS lookups extremely fast.

C. DNS Pre-fetching

The DNS infrastructure can use **pre-fetching** for frequently accessed records.

When a cached record is close to expiration, the resolver can refresh the record before it expires, where supported by the DNS infrastructure.

For frequently accessed application domains, this reduces the possibility of users encountering a cache miss and having to wait for a complete DNS lookup.

D. TTL Optimization

TTL, or **Time To Live**, determines how long a DNS record can remain cached.

The company should avoid setting TTL values unnecessarily low because this creates frequent cache misses.

For relatively stable records, a longer TTL can be used. For records that need frequent changes, a shorter TTL is safer.

For example:

DNS Record	Suggested Approach
Stable application endpoint	Longer TTL
Frequently changed service	Shorter TTL
Failover-related records	Carefully selected TTL
CDN/static content records	Longer TTL where appropriate

The exact TTL should be selected according to how frequently the company's infrastructure changes.

E. Redundant Authoritative Servers

The company should not depend on one DNS server or one data center.

Multiple authoritative DNS servers should be deployed across different regions and network providers.

If one DNS location fails, requests can still be answered by another location. This provides **continuous service availability**.

F. Health Monitoring and Automatic Failover

The DNS infrastructure should continuously monitor the health of application endpoints.

If a particular server or region becomes unavailable, DNS traffic can be redirected to a healthy endpoint.

This is especially important for a multinational company because a failure in one continent should not bring down the service globally.

3. How the Architecture Can Reduce Lookup Time Below 50 ms

The target of **less than 50 ms** should be achieved mainly by reducing the number of DNS requests that require a fresh recursive lookup.

The proposed approach works as follows:

Step 1: User requests the application domain.

Step 2: The request reaches a geographically suitable Anycast DNS service.

Step 3: The recursive resolver checks its cache.

Step 4: If the record is cached and still valid, the resolver immediately returns the IP address.

Step 5: If the record is approaching expiration, pre-fetching can refresh it where supported.

Step 6: If the record is not cached, the resolver contacts the appropriate authoritative DNS server.

With geographically distributed DNS servers, caching, and proper TTL values, most users should receive cached DNS responses quickly. However, the company should **measure the result from each major region**, because a guaranteed sub-50-ms lookup time for every user cannot be promised simply by deploying Anycast.

4. Advantages of the Proposed Architecture

1. Better Performance

Anycast reduces the distance between users and DNS infrastructure. Caching further reduces the need for repeated DNS resolution.

2. Lower DNS Latency

Users can be directed to a nearby DNS service, while cached responses can be returned much faster than a complete recursive lookup.

3. High Availability

Multiple DNS servers ensure that failure of one server does not completely stop DNS resolution.

4. Better Scalability

As the company expands into new countries, additional DNS locations can be introduced without redesigning the entire system.

5. Improved Fault Tolerance

A failure in one DNS location or network path can be handled by other available locations.

6. Reduced DNS Traffic

Caching and suitable TTL values reduce repeated queries to authoritative servers.

5. Limitations of the Proposed Architecture

Although the architecture provides significant benefits, it also has some limitations.

1. Higher Cost

Deploying DNS infrastructure across several continents requires additional servers, network connectivity, monitoring, and maintenance.

2. Anycast Routing Complexity

Anycast depends on Internet routing. The physically closest server is not always the server selected by routing, so "nearest" does not always mean geographically closest.

3. TTL Trade-off

A long TTL improves caching and performance but can delay DNS changes from reaching users. A very short TTL allows faster changes but creates more DNS queries.

4. Pre-fetching Is Not a Complete Solution

Pre-fetching helps popular records, but it does not eliminate delays for new or rarely requested domains.

5. Monitoring Becomes More Important

With multiple DNS locations, the company must monitor server health, routing, response times, and synchronization. A configuration problem can affect many users if it is distributed globally.

Comparison of the Proposed Approach

Factor	Current System	Proposed System
DNS lookup time	Around 800 ms	Target below 50 ms for typical/cached lookups
Server location	May be distant	Globally distributed
DNS caching	Possibly inefficient	Optimized caching
Failure handling	Limited	Multiple redundant DNS locations
Scalability	Moderate	High
Availability	Can be affected by server failure	High availability through redundancy
DNS traffic	Higher due to cache misses	Reduced through caching and TTL optimization

Conclusion

The **800 ms DNS lookup time** is a major reason for the delay experienced when users open the company's application. The problem can be caused by a combination of geographical distance, cache misses, overloaded DNS servers, network conditions, and unsuitable TTL settings.

A better solution is to build a **globally distributed Anycast DNS architecture** with redundant authoritative servers. DNS caching should handle frequently requested records, pre-fetching can refresh popular records before expiration where supported, and TTL values should be selected according to the stability of each DNS record.

This architecture improves **performance, scalability, and availability** at the same time. Although it requires additional cost and management, it is suitable for a multinational organization because DNS service can continue even when a particular server or region experiences a failure.

Overall, the goal should be to make the **common cached DNS lookup path comfortably below 50 ms**, while maintaining redundancy and monitoring so that DNS failures do not affect the company's global web services.

3. Analysis and Redesign of DNS Infrastructure

Introduction

A multinational software company provides web services to users and employees located across different continents. However, users are experiencing a noticeable delay before the application homepage starts loading. The network analysis shows that the **average DNS lookup time is around 800 ms**, which is very high for a web application.

DNS is responsible for converting a domain name into the IP address of the server hosting the application. If DNS resolution itself takes 800 ms, the browser has to wait before it can even start communicating with the web server. Therefore, improving the DNS infrastructure can significantly improve the initial page-loading experience.

The proposed solution is to combine **Anycast DNS, DNS caching and pre-fetching, and proper TTL optimization**, along with redundant DNS servers distributed across different geographical locations.

1. Reasons for the High DNS Resolution Time

There can be several reasons for the unusually high DNS lookup time.

1. Geographical Distance

The company operates across multiple continents. If users in Asia are querying a DNS server located in North America, the request has to travel a long physical distance. This increases network round-trip time.

2. Slow or Distant Recursive DNS Resolver

The user's DNS request may be handled by a recursive resolver that is overloaded or geographically far away. The resolver may need to contact several DNS servers before obtaining the final IP address.

3. Multiple DNS Queries

DNS resolution may involve several steps:

Client → Recursive Resolver → Root Server → TLD Server → Authoritative DNS Server

If the required information is not already cached, each step can add some delay.

4. Low or Poorly Selected TTL Values

If DNS records have very short TTL values, cached records expire frequently. This forces recursive DNS servers to perform fresh lookups more often, increasing DNS traffic and resolution time.

5. DNS Server Overload

If the company's authoritative DNS servers receive a large number of requests and have limited capacity, response times can increase, particularly during peak periods.

6. Network Congestion and Packet Loss

DNS queries use network communication like any other service. Congestion, packet loss, or retransmissions can delay DNS responses.

Therefore, the **800 ms average lookup time is probably not caused by a single issue**. It is more likely the result of geographical distance, poor DNS server placement, cache misses, and overloaded or inefficient DNS infrastructure.

2. Proposed DNS Architecture

The company should use a **globally distributed, redundant DNS architecture based mainly on Anycast DNS**, supported by DNS caching, pre-fetching, and suitable TTL values.

Proposed Architecture

User Device



Nearest Anycast DNS Resolver



Recursive DNS Cache



Distributed Authoritative DNS Servers



Application Server

The DNS infrastructure should have servers in major regions such as **Asia, Europe, North America, and other important user locations**.

A. Anycast DNS

In Anycast DNS, multiple DNS servers located at different geographical locations use the same IP address.

For example:

- DNS Server A → Asia
- DNS Server B → Europe
- DNS Server C → North America
- DNS Server D → Australia

All of them advertise the same Anycast address.

When a user sends a DNS request, network routing normally directs the request toward an appropriate nearby/low-cost Anycast location.

This reduces the physical distance between the user and the DNS service and can significantly reduce lookup latency.

B. DNS Caching

Caching should be used at recursive DNS resolvers.

Once a DNS record is obtained, the resolver can store it temporarily. If another employee requests the same domain before the record expires, the resolver can return the cached answer instead of performing the entire DNS resolution process again.

This can make subsequent DNS lookups extremely fast.

C. DNS Pre-fetching

The DNS infrastructure can use **pre-fetching** for frequently accessed records.

When a cached record is close to expiration, the resolver can refresh the record before it expires, where supported by the DNS infrastructure.

For frequently accessed application domains, this reduces the possibility of users encountering a cache miss and having to wait for a complete DNS lookup.

D. TTL Optimization

TTL, or **Time To Live**, determines how long a DNS record can remain cached.

The company should avoid setting TTL values unnecessarily low because this creates frequent cache misses.

For relatively stable records, a longer TTL can be used. For records that need frequent changes, a shorter TTL is safer.

For example:

DNS Record	Suggested Approach
-------------------	---------------------------

Stable application endpoint	Longer TTL
-----------------------------	------------

Frequently changed service	Shorter TTL
----------------------------	-------------

Failover-related records	Carefully selected TTL
--------------------------	------------------------

CDN/static content records	Longer TTL where appropriate
----------------------------	------------------------------

The exact TTL should be selected according to how frequently the company's infrastructure changes.

E. Redundant Authoritative Servers

The company should not depend on one DNS server or one data center.

Multiple authoritative DNS servers should be deployed across different regions and network providers.

If one DNS location fails, requests can still be answered by another location. This provides **continuous service availability**.

F. Health Monitoring and Automatic Failover

The DNS infrastructure should continuously monitor the health of application endpoints.

If a particular server or region becomes unavailable, DNS traffic can be redirected to a healthy endpoint.

This is especially important for a multinational company because a failure in one continent should not bring down the service globally.

3. How the Architecture Can Reduce Lookup Time Below 50 ms

The target of **less than 50 ms** should be achieved mainly by reducing the number of DNS requests that require a fresh recursive lookup.

The proposed approach works as follows:

Step 1: User requests the application domain.

Step 2: The request reaches a geographically suitable Anycast DNS service.

Step 3: The recursive resolver checks its cache.

Step 4: If the record is cached and still valid, the resolver immediately returns the IP address.

Step 5: If the record is approaching expiration, pre-fetching can refresh it where supported.

Step 6: If the record is not cached, the resolver contacts the appropriate authoritative DNS server.

With geographically distributed DNS servers, caching, and proper TTL values, most users should receive cached DNS responses quickly. However, the company should **measure the result from each major region**, because a guaranteed sub-50-ms lookup time for every user cannot be promised simply by deploying Anycast.

4. Advantages of the Proposed Architecture

1. Better Performance

Anycast reduces the distance between users and DNS infrastructure. Caching further reduces the need for repeated DNS resolution.

2. Lower DNS Latency

Users can be directed to a nearby DNS service, while cached responses can be returned much faster than a complete recursive lookup.

3. High Availability

Multiple DNS servers ensure that failure of one server does not completely stop DNS resolution.

4. Better Scalability

As the company expands into new countries, additional DNS locations can be introduced without redesigning the entire system.

5. Improved Fault Tolerance

A failure in one DNS location or network path can be handled by other available locations.

6. Reduced DNS Traffic

Caching and suitable TTL values reduce repeated queries to authoritative servers.

5. Limitations of the Proposed Architecture

Although the architecture provides significant benefits, it also has some limitations.

1. Higher Cost

Deploying DNS infrastructure across several continents requires additional servers, network connectivity, monitoring, and maintenance.

2. Anycast Routing Complexity

Anycast depends on Internet routing. The physically closest server is not always the server selected by routing, so "nearest" does not always mean geographically closest.

3. TTL Trade-off

A long TTL improves caching and performance but can delay DNS changes from reaching users. A very short TTL allows faster changes but creates more DNS queries.

4. Pre-fetching Is Not a Complete Solution

Pre-fetching helps popular records, but it does not eliminate delays for new or rarely requested domains.

5. Monitoring Becomes More Important

With multiple DNS locations, the company must monitor server health, routing, response times, and synchronization. A configuration problem can affect many users if it is distributed globally.

Comparison of the Proposed Approach

Factor	Current System	Proposed System
DNS lookup time	Around 800 ms	Target below 50 ms for typical/cached lookups
Server location	May be distant	Globally distributed
DNS caching	Possibly inefficient	Optimized caching
Failure handling	Limited	Multiple redundant DNS locations
Scalability	Moderate	High
Availability	Can be affected by server failure	High availability through redundancy
DNS traffic	Higher due to cache misses	Reduced through caching and TTL optimization

Conclusion

The **800 ms DNS lookup time** is a major reason for the delay experienced when users open the company's application. The problem can be caused by a combination of geographical distance, cache misses, overloaded DNS servers, network conditions, and unsuitable TTL settings.

A better solution is to build a **globally distributed Anycast DNS architecture** with redundant authoritative servers. DNS caching should handle frequently requested records, pre-fetching can refresh popular records before expiration where supported, and TTL values should be selected according to the stability of each DNS record.

This architecture improves **performance, scalability, and availability** at the same time. Although it requires additional cost and management, it is suitable for a multinational organization because DNS service can continue even when a particular server or region experiences a failure.

Overall, the goal should be to make the **common cached DNS lookup path comfortably below 50 ms**, while maintaining redundancy and monitoring so that DNS failures do not affect the company's global web services.

4. TCP Performance Analysis in a High-Frequency Stock Trading Platform

Introduction

A financial institution runs an electronic stock trading platform where thousands of buy and sell orders are received every second. Under normal conditions, an order acknowledgment takes about **1 ms**, but during market opening hours the latency rises to **40 ms**.

For a stock trading system, even a few milliseconds can matter because customers expect their orders and confirmations to be processed quickly. The increase in TCP retransmissions, longer connection establishment time, and excessive buffering suggests that the TCP configuration and the large number of simultaneous connections are contributing to the problem.

Three important TCP-related factors are **flow control, Nagle's algorithm, and SYN queue limitations**.

1. TCP Factors Responsible for the Latency Increase

A. TCP Flow Control

TCP uses a **sliding window mechanism** to control how much data can be sent before receiving acknowledgments. The receiver advertises a receive window indicating how much additional data it can accept.

During market opening, thousands of orders may arrive almost simultaneously. If the receiving application or server cannot process the incoming data quickly enough, its receive buffer can become full. The advertised TCP window may then become smaller.

This causes the sender to slow down or temporarily wait for acknowledgments. As a result, order messages can spend more time waiting in buffers, increasing the acknowledgment latency.

Simple flow:

High order rate → Receive buffer fills → TCP window decreases → Sender slows down → Orders wait longer → Latency increases

B. Nagle's Algorithm

Nagle's algorithm is designed to reduce the number of small TCP packets. Instead of immediately transmitting every small piece of data, TCP may temporarily hold small segments and combine them with other data.

This is useful for reducing network overhead, but it is not ideal for a stock trading application where messages may be very small and every millisecond matters.

For example, a trading application may send a small order acknowledgment. If Nagle's algorithm holds that data while waiting for an acknowledgment or more data, the customer may receive the confirmation later than expected.

Therefore, **Nagle's algorithm can introduce unnecessary delay for small, latency-sensitive messages.**

C. SYN Queue Limitations

Before a TCP connection can transfer data, the client and server perform the **three-way handshake**:

SYN → SYN-ACK → ACK

During market opening, many clients or trading components may attempt to establish connections at almost the same time. The server maintains a queue for partially established connections.

If the **SYN queue becomes full**, new connection requests may be delayed, retransmitted, or dropped. This explains the observed increase in connection establishment time.

Repeated connection attempts can also create additional traffic and processing overhead, making the situation worse during peak periods.

2. Effect of Each Factor on High-Frequency Transaction Processing

TCP Factor	What Happens	Effect on Trading
Flow control	Receiver advertises a smaller window when buffers are busy	Sender slows down and orders wait
Nagle's algorithm	Small packets may be temporarily buffered	Order acknowledgments can be delayed
SYN queue limitation	Too many connection requests compete for limited queue space	Connection establishment becomes slower
TCP retransmissions	Lost packets are sent again	Increases delay and processing overhead

Flow Control in Trading

In a high-frequency system, thousands of transactions may arrive within a very short period. If the receiving application processes them more slowly than they arrive, TCP flow control prevents the sender from overwhelming the receiver.

Although this protects the system from buffer overflow, it can increase latency. In a trading environment, excessive buffering is especially undesirable because an order that is waiting in a queue is not being processed immediately.

Nagle's Algorithm in Trading

Trading messages are often small. An order may contain only a limited amount of information such as the stock symbol, quantity, price, and transaction type.

If small messages are delayed so that they can be combined into larger packets, the network may become more efficient, but the application loses some of its speed advantage.

For a normal application, a small delay may not matter. For electronic trading, however, unnecessary milliseconds can affect the customer's experience and order processing time.

SYN Queue Limitations in Trading

If the system creates new TCP connections frequently instead of maintaining existing connections, market opening can produce a sudden burst of SYN requests.

When the SYN queue is unable to handle the burst, connection establishment becomes slower. This is particularly harmful because the trading request cannot even begin until the TCP connection is established.

For this reason, a trading platform should avoid unnecessary connection creation and should preferably use **persistent connections** where appropriate.

3. Recommended TCP Configuration Changes

1. Disable Nagle's Algorithm for Latency-Sensitive Connections

For trading messages that require immediate transmission, **TCP_NODELAY** can be enabled.

This prevents Nagle's algorithm from intentionally delaying small TCP segments.

The benefit is faster delivery of small order and acknowledgment messages.

However, it should be applied selectively because disabling Nagle everywhere can increase the number of packets and network overhead.

2. Increase and Properly Tune TCP Receive Buffers

The server should have sufficient receive-buffer capacity to handle bursts of incoming orders.

Increasing the TCP receive buffer can prevent the advertised receive window from becoming unnecessarily small.

However, simply making the buffers extremely large is not a complete solution because excessive buffering can itself increase latency. The buffers should be sized according to the expected traffic and application processing rate.

3. Improve Application Processing Speed

TCP flow control often reflects a problem at the receiver. Therefore, the institution should also optimize the trading application so that received orders are processed quickly.

The system should avoid situations where the TCP receive buffer fills because the application cannot consume incoming data fast enough.

4. Increase SYN Backlog/Queue Capacity

The server's SYN backlog should be configured to handle the expected burst of connection attempts during market opening.

This provides more room for pending connections and reduces the possibility of new connection requests being dropped or repeatedly retransmitted.

The exact value should be selected according to the operating system and expected workload rather than simply setting an unnecessarily large number.

5. Use Persistent TCP Connections

Instead of opening a new TCP connection for every order, the trading platform should maintain established connections whenever the architecture allows it.

This avoids repeatedly performing the:

SYN → SYN-ACK → ACK

handshake and reduces connection establishment latency.

6. Monitor and Reduce Retransmissions

The network team should investigate the reason for the increased TCP retransmissions. Possible causes include packet loss, congestion, overloaded network devices, or insufficient capacity.

Reducing packet loss is important because retransmissions directly add delay to reliable TCP communication.

7. Use TCP Keep-Alive Carefully

Persistent connections can be maintained using appropriate keep-alive mechanisms. This allows the system to detect dead connections without unnecessarily creating new connections for every transaction.

Recommended Configuration Approach

The trading platform should use the following approach:

High-frequency order traffic

↓

Persistent TCP connections

↓

TCP_NODELAY for latency-sensitive messages

↓

Properly sized receive/send buffers

↓

Adequate SYN backlog

↓

Fast application processing

↓

Reduced retransmissions and faster acknowledgment

This approach addresses both the TCP-level delays and the application behavior causing excessive buffering.

Advantages of the Recommended Changes

The proposed changes can provide several benefits:

- Faster order acknowledgment
- Reduced delay for small trading messages
- Fewer unnecessary connection handshakes
- Better handling of sudden market-opening traffic
- Reduced buffering
- Improved connection reliability
- Better utilization of existing TCP connections
- More consistent latency during peak periods

Important Consideration

It is important not to change every TCP parameter blindly. For example, simply increasing buffer sizes may solve one problem but create another problem called **bufferbloat**, where excessive data remains queued and increases latency.

Similarly, disabling Nagle's algorithm improves responsiveness for small messages but can increase packet overhead. Therefore, the configuration should be tested under a realistic market-opening workload before being deployed.

Conclusion

The increase in order acknowledgment latency from **1 ms to 40 ms** can be explained by several TCP-related issues occurring together. **Flow control** can slow the sender when receiving buffers become congested, **Nagle's algorithm** can delay small trading messages, and **SYN queue limitations** can increase connection establishment time when thousands of connections are attempted simultaneously.

For a high-frequency trading platform, the recommended approach is to use **TCP_NODELAY for latency-sensitive traffic, properly tune TCP buffers, increase the SYN backlog according to workload, use persistent connections, and investigate the causes of retransmissions.**

The main goal should not simply be to make TCP faster, but to prevent unnecessary waiting at every stage of the transaction. With careful TCP tuning and efficient application processing, the platform can reduce the 40 ms peak latency and provide faster and more reliable trade confirmations.